

Teórica 6
UML – Diagramas de Classes

Por que é que os programadores não gostam da natureza?

Herança e polimorfismo

Herança

Um mecanismo para derivar novas classes a partir de classes existentes.

Uma subclasse derivada de uma classe existente (= superclasse) herda todos os atributos e operações visíveis (especificação e implementação) da superclasse.

Uma subclasse pode:

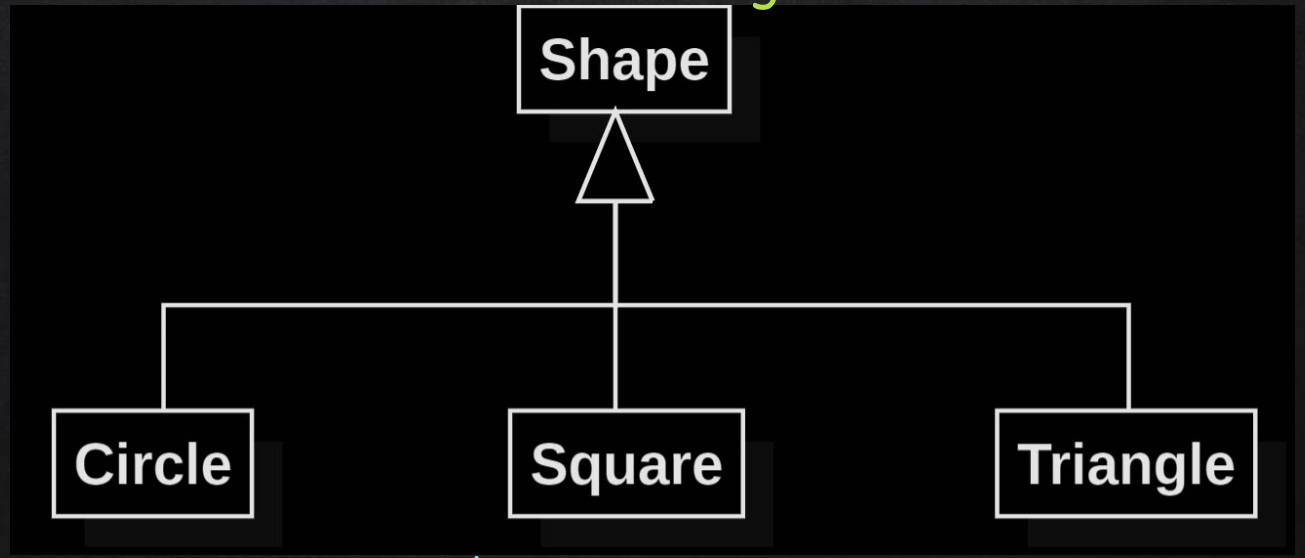
- Definir novos atributos (formalmente propriedades) e/ou operações
- Reescrever a implementação de operações herdadas
- Adicionar o seu próprio código às operações herdadas

Generalização é uma relação entre uma classe mais específica e uma classe mais geral

- A classe mais específica é totalmente consistente com a classe mais geral
- As duas classes obedecem ao princípio de substituição de Liskov
  - Podemos usar a classe mais específica em qualquer lugar onde a classe mais geral é utilizada, sem introduzir erros no sistema
  - Este é um tipo de dependência muito mais forte do que a dependência por meio de associações de classes
  - Este mecanismo leva ao mais alto nível de acoplamento entre classes

Criamos uma hierarquia de generalização generalizando a partir de classes mais específicas e especializando a partir de classes mais gerais

elemento mais geral

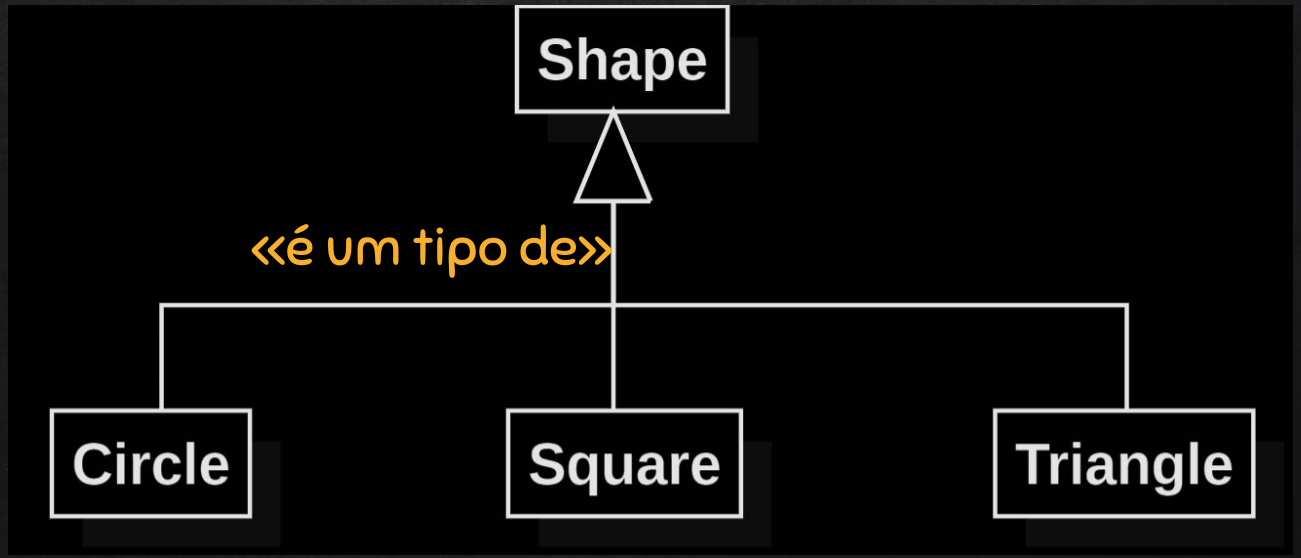


Generalização

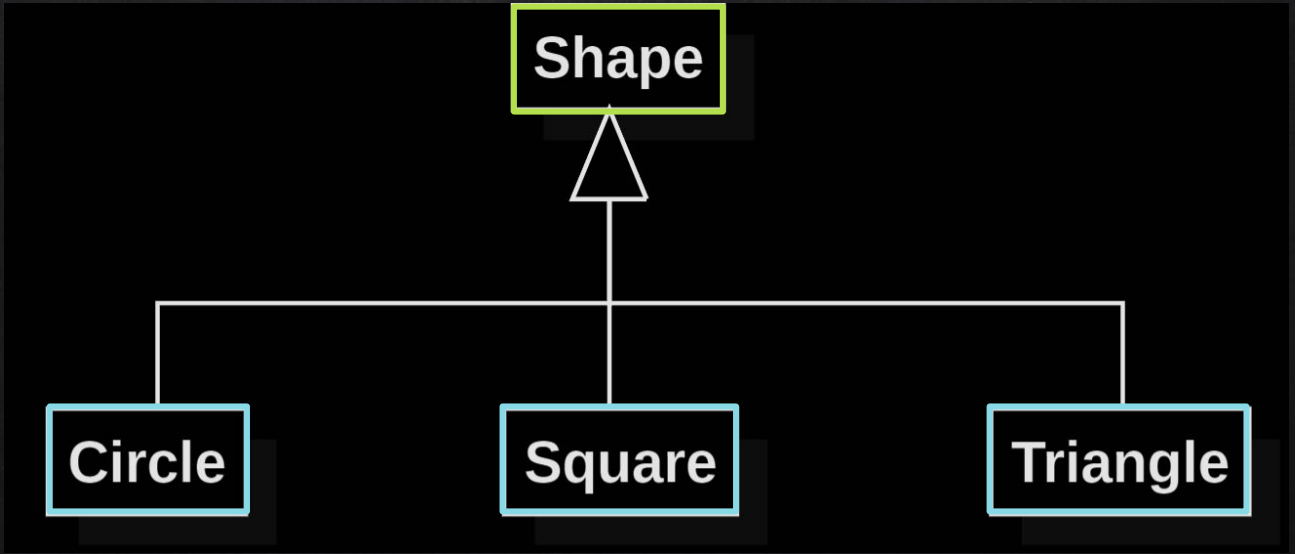
Especialização

elementos mais específicos

Um círculo é um tipo de forma
Um quadrado é um tipo de forma
Um triângulo é um tipo de forma



Uma forma é uma classe pai / superclasse / classe base / ascendente
Um círculo / quadrado / triângulo
é uma classe filha / subclasse / descendente

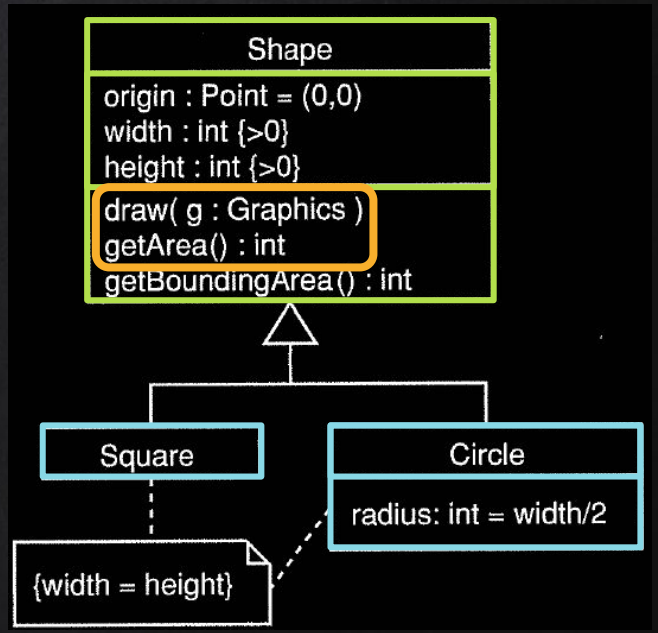


Generalização e herança

- As classes podem estar hierarquicamente organizadas, onde a superclasse é a generalização de uma ou mais classes (subclasses).
- Uma subclasse herda os atributos, operações, relações e dependências da superclasse
- A subclasse pode adicionar novos recursos e substituir outros recursos
- A generalização em UML é implementada como herança em POO

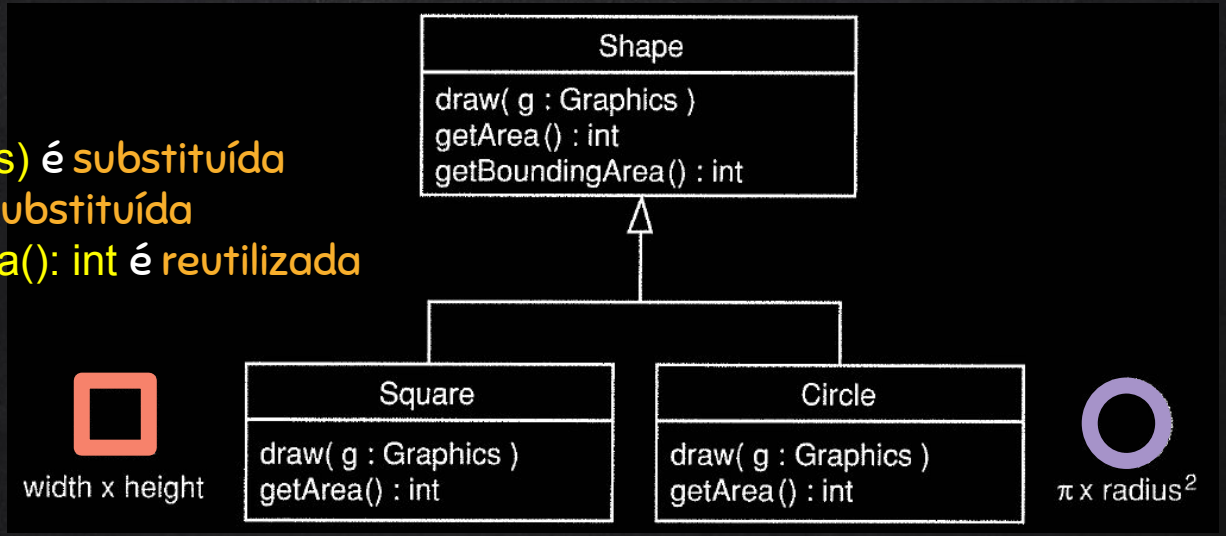
Substituição e por que podemos precisar dela...

- `Square` e `circle` são tipos de `shape`
- Herdam características, relações e restrições da forma
- Os elementos herdados não são visíveis no diagrama (são implícitos!)
- No entanto, operações como `draw()` e `getArea()` não podem ser definidas de forma geral para `shape`.



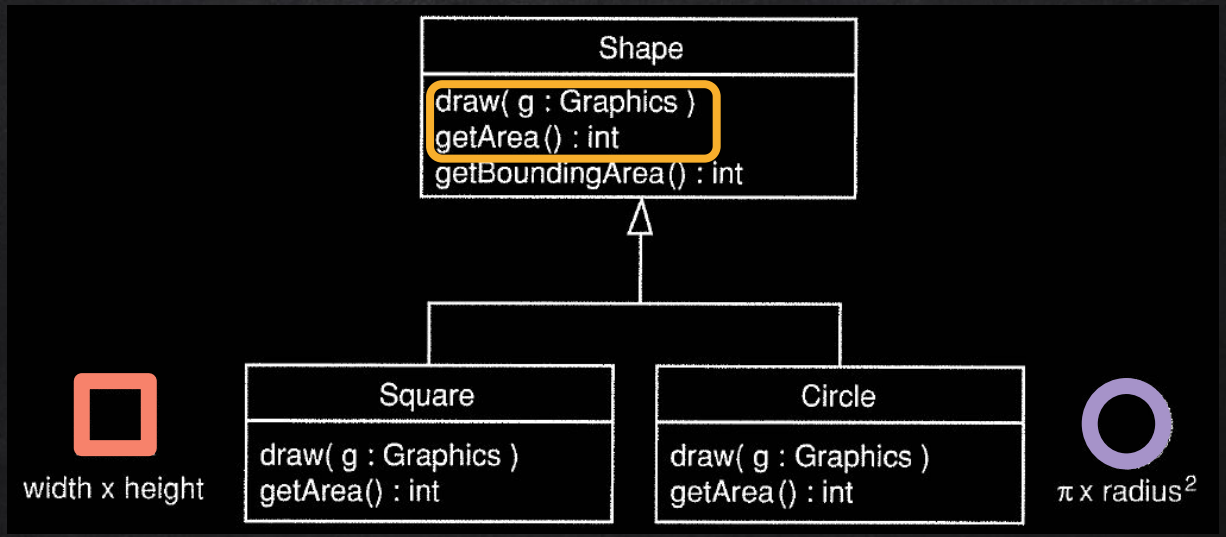
Subclasses substituem as operações herdadas, oferecendo uma nova operação com a mesma assinatura

draw(g: Graphics) é substituída
getArea(): int é substituída
getBoundingArea(): int é reutilizada como está



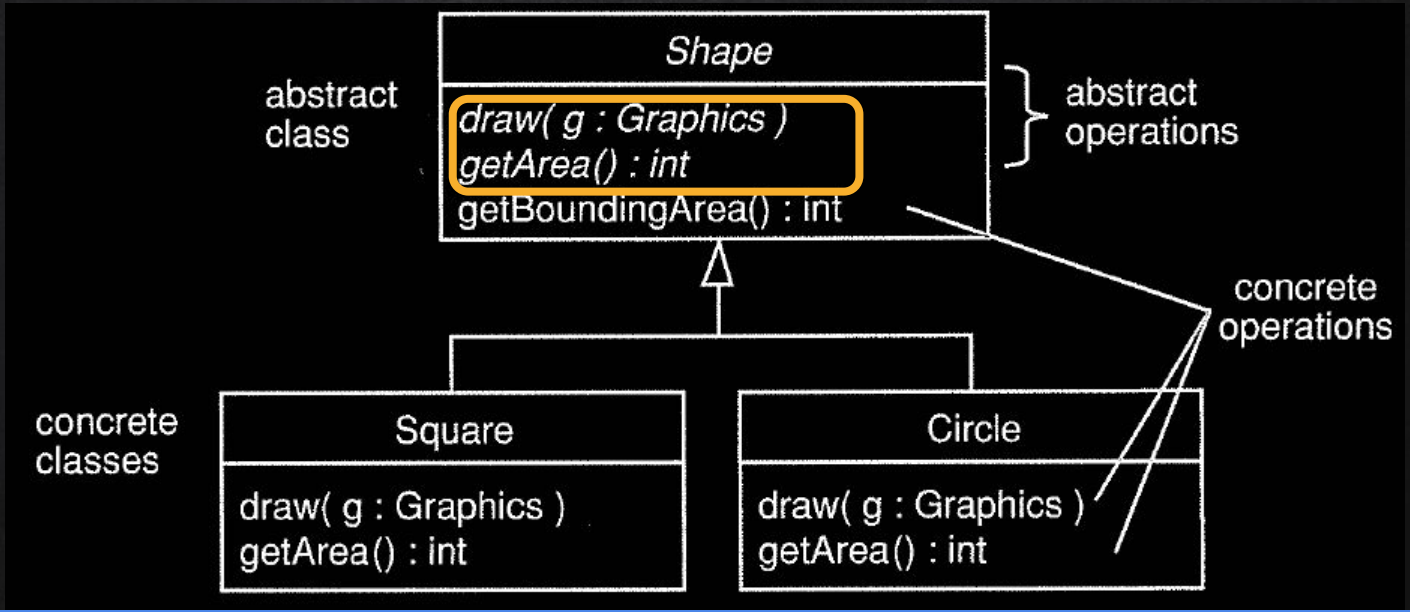
Aviso: linguagens de programação como C++ e Java não incluem o tipo de retorno na assinatura da operação. Se redefinir o tipo de retorno para um tipo não diferente, obterá um erro do compilador ou interpretador.

Na verdade, queremos mesmo criar objectos Shape?
Nem por isso. Como implementaríamos apenas alguns desses métodos?



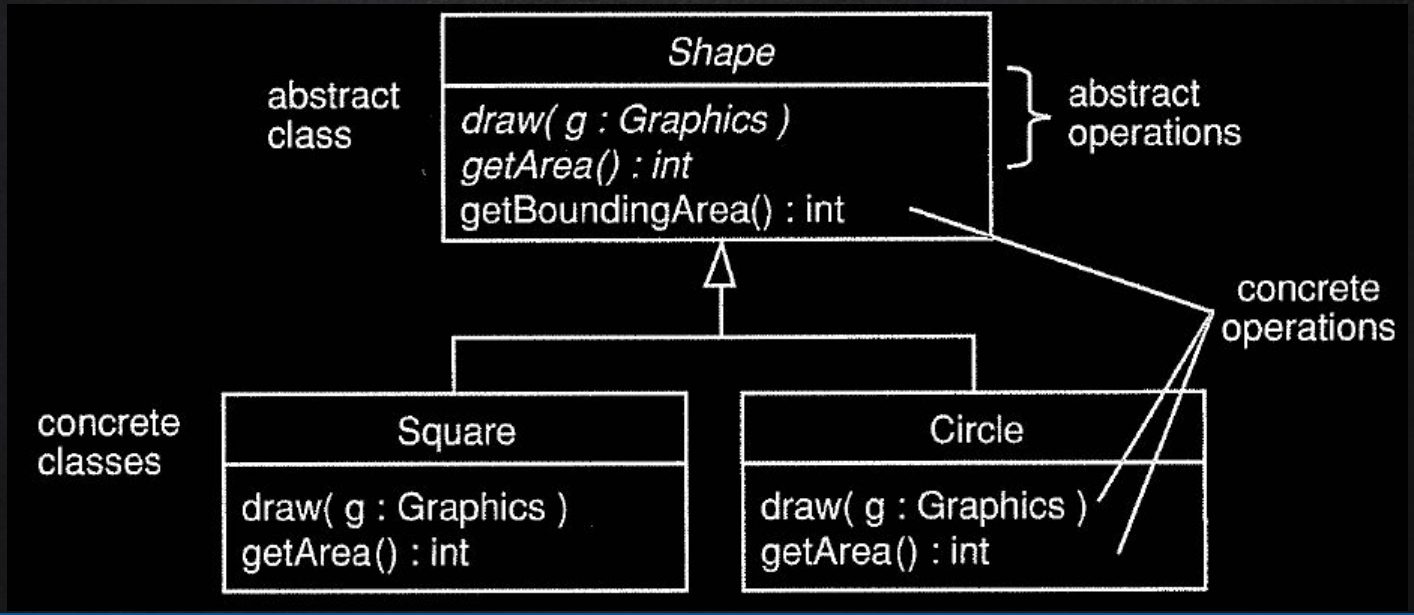
Quando uma superclasse foi criada para factorizar propriedades e comportamentos comuns de um conjunto de classes, mas contém uma ou mais operações que sabemos que temos de redefinir de alguma forma, devemos usar uma *classe abstracta*!

As operações abstractas não têm implementação:
A implementação é adiada para operações nas subclasses



Repare como as assinaturas *draw(g: Graphics)* e *getArea()* estão *em itálico*. A notação *em itálico* nos diagramas de classes UML representa o modificador abstracto!

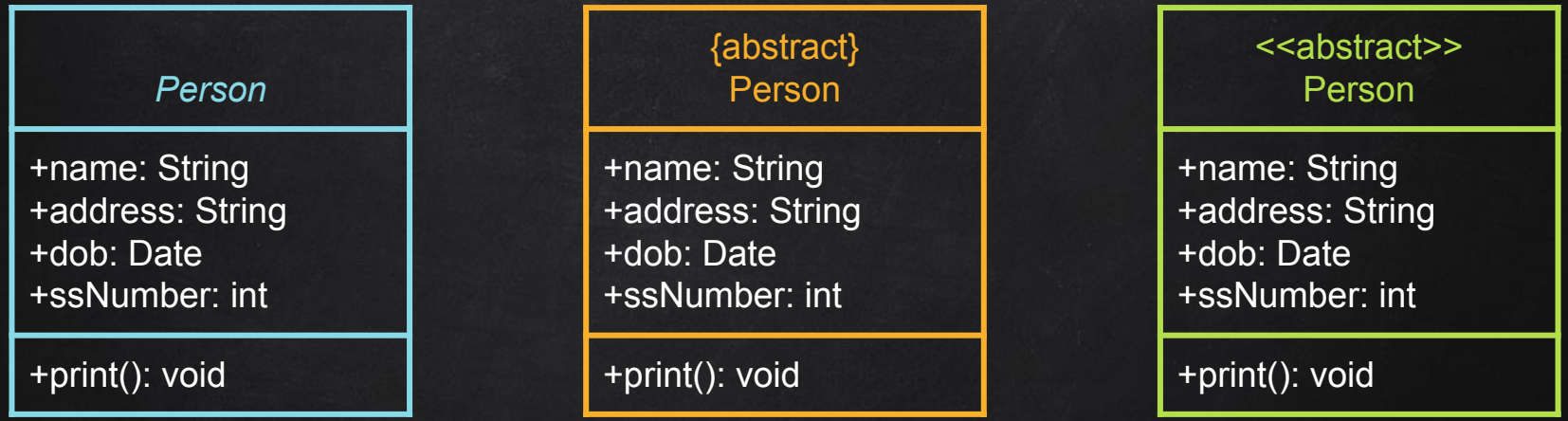
As classes abstractas incluem uma ou mais operações abstractas e não podem ser instanciadas



Uma classe abstracta é indicada por um dos seguintes elementos:

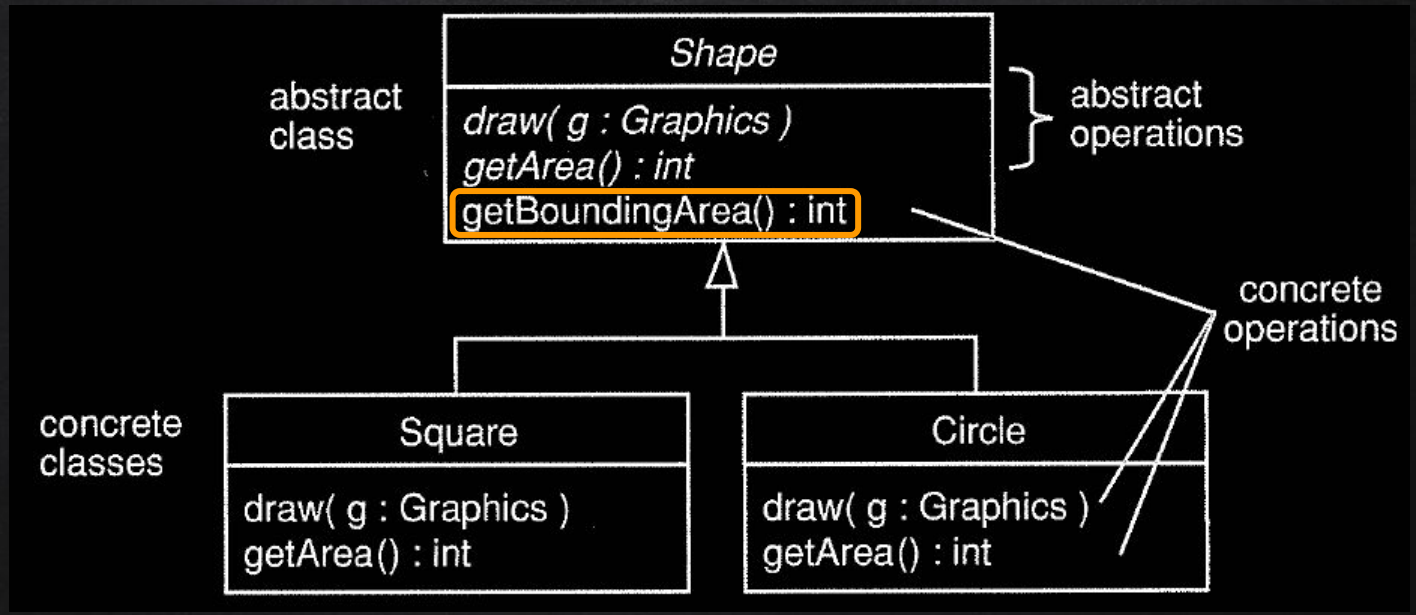
o seu nome em *itálico*, o estereótipo `<<abstract>>` ou o valor marcado `{abstract}`

Declarando uma classe abstracta usando a notação em itálico, uma tag ou um estereótipo



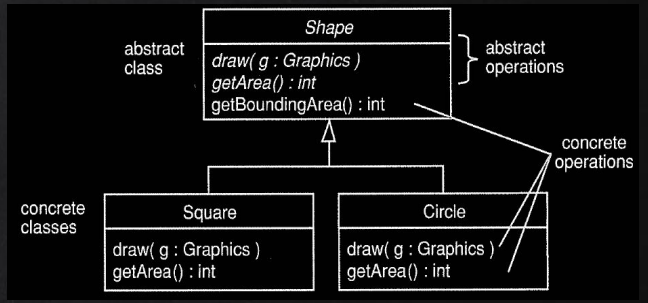
Estas definições representam exactamente a mesma semântica:
Person é uma classe abstracta.

getBoundingArea() é concreta porque é sempre calculada da mesma forma (largura * altura)

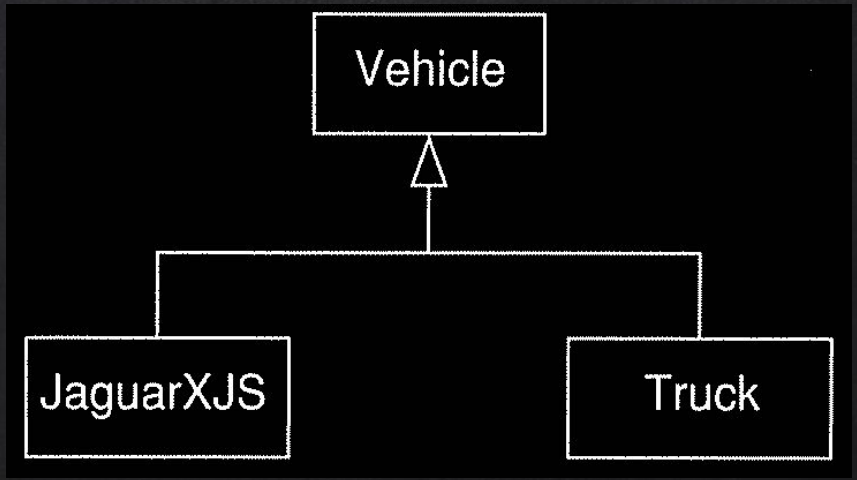


Vantagens de usar classes e operações abstractas

- O conjunto de operações abstractas na classe abstracta devem ser implementadas nas subclasses concretas
- Pode escrever código para manipular Shapes e depois substituir Square ou Circle, conforme apropriado
- O código escrito para a classe abstracta deve funcionar correctamente para todas as classes concretas – lembre-se do princípio de substituição!



Mantenha as classes no mesmo nível hierárquico e no mesmo nível de abstração



Neste caso, o Jaguar XJS é um modelo de uma determinada marca de automóveis, enquanto que Truck é um veículo definido num nível de abstração diferente.

Esta é uma má prática.

Polimorfismo

Polimorfismo

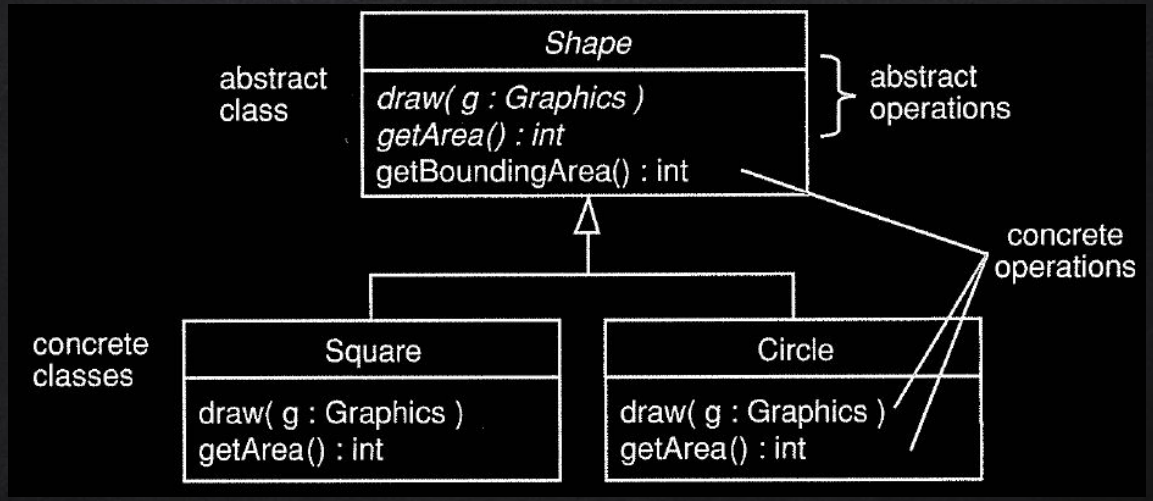
Capacidade de assumir diferentes formas.

Durante a execução de um programa, um atributo polimórfico pode ter referências a objectos de diferentes classes.

Polimorfismo

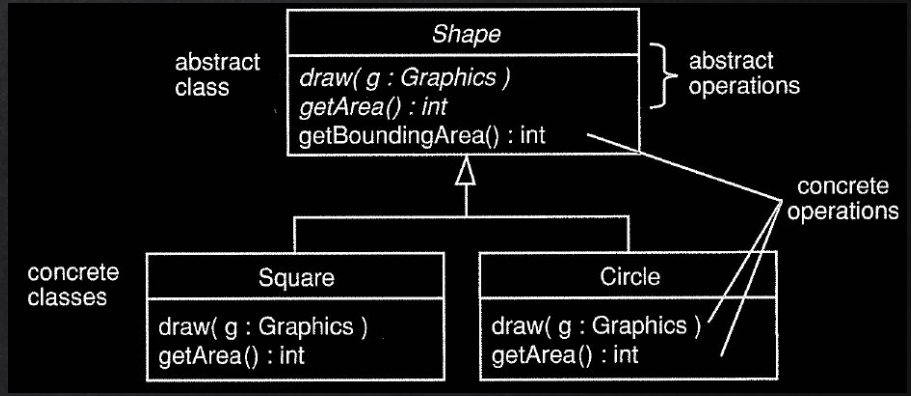
- polimorfismo paramétrico, mais conhecido como genericidade – são utilizados parâmetros de tipo
  - Em Java, por exemplo, as classes concretas são transferidas para as operações como argumentos
- polimorfismo de inclusão
  - as operações podem ser aplicadas às classes e às suas subclasses directas e indirectas
- sobrecarga de operações
  - as operações podem ter o mesmo nome, mas assinaturas diferentes
- coerção – conversão de tipos
  - O tipo de destino da conversão deve estar em conformidade com o tipo de origem

Uma operação polimórfica é uma operação com várias implementações



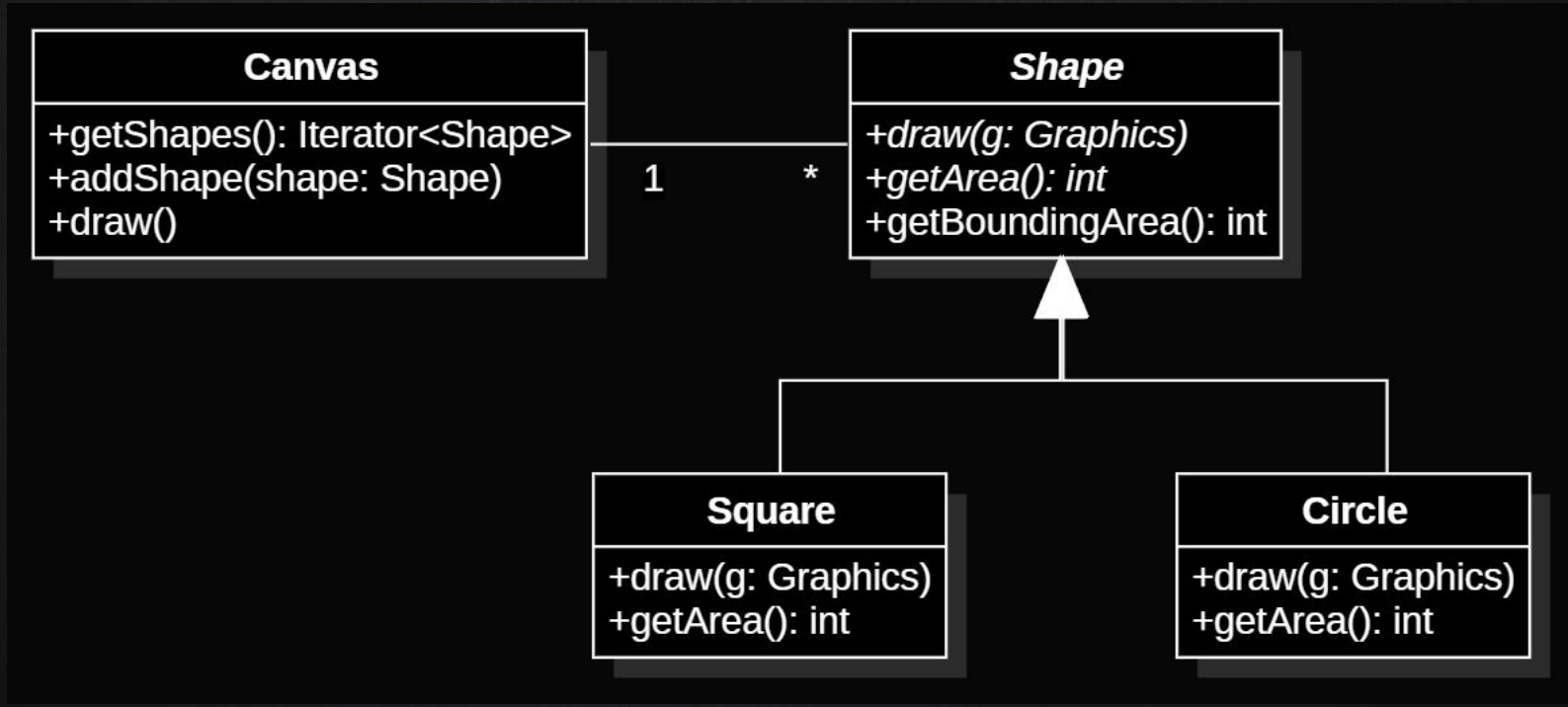
Objectos de diferentes subclasses recebem exatamente a mesma mensagem, mas reagem de forma diferente, de acordo com a sua implementação da operação que foi chamada

- `Square` e `Circle` especializam `Shape`
- Devem oferecer implementações para as operações abstractas `draw()` e `getArea()`

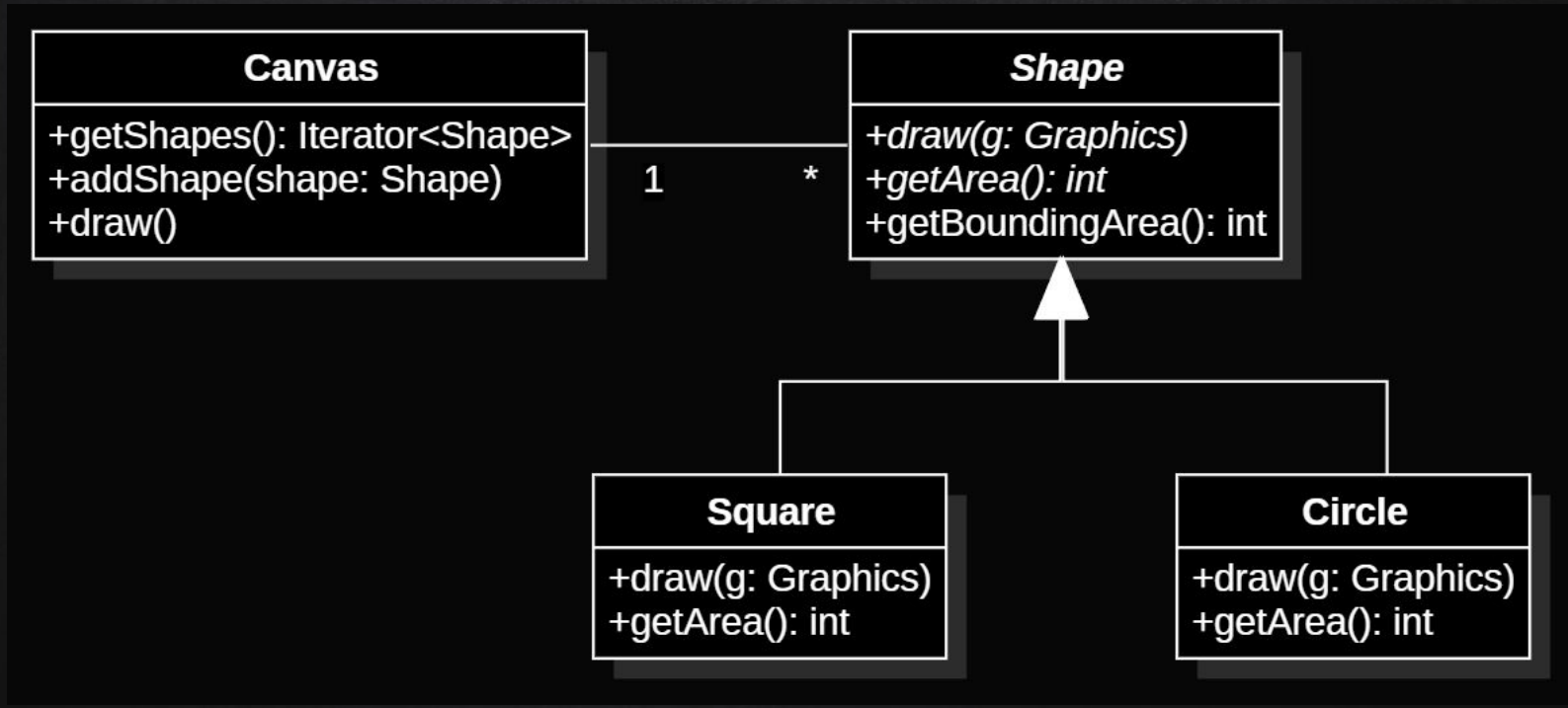


As classes concretas devem oferecer implementações concretas para TODAS as operações abstractas herdadas das suas superclasses abstractas
Objectos de diferentes subclasses implementam a operação abstracta herdada de formas diferentes – daí o termo polimorfismo

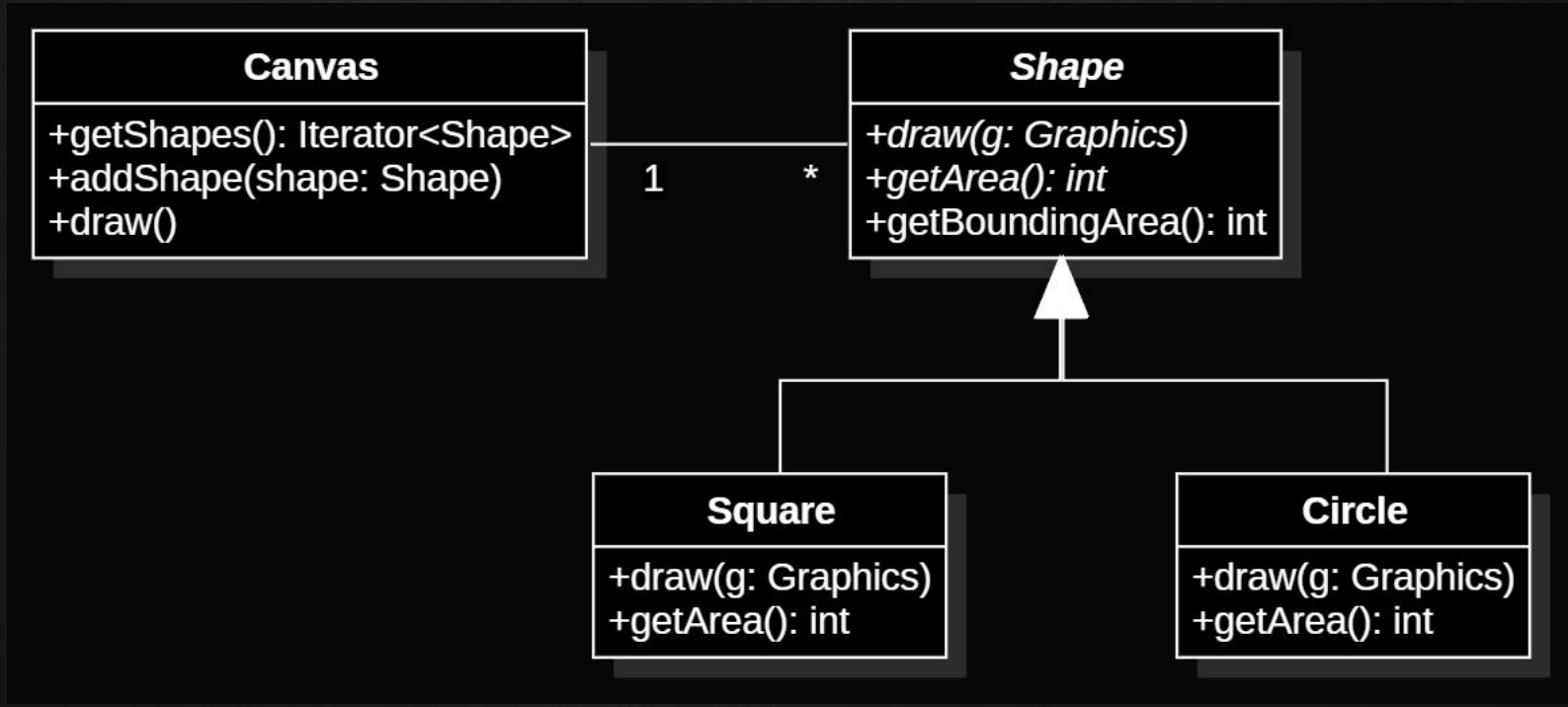
Cada elemento da colecção Shapes no Canvas pertence a uma subclasse



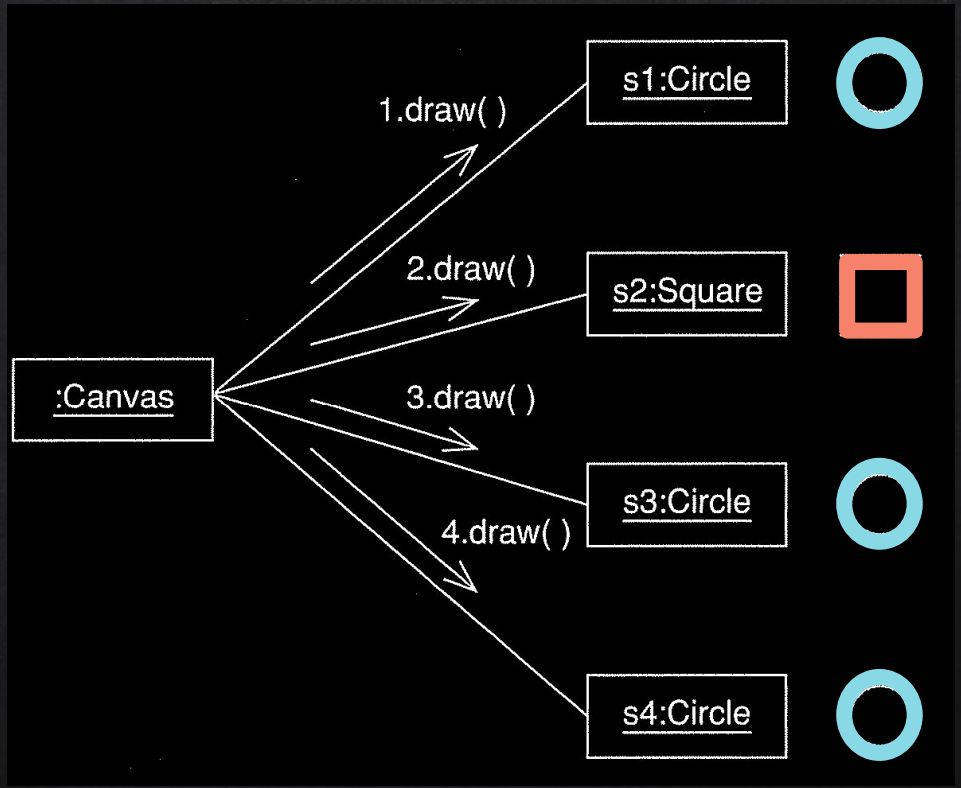
Ao chamar uma operação draw() ou getArea() de *Shape* a partir do Canvas, não sabemos que tipo de *Shape* irá responder



A classe que responderá à chamada será determinada por meio de um mecanismo chamado ligação dinâmica



Cada membro da colecção utilizará a sua própria implementação da operação draw()



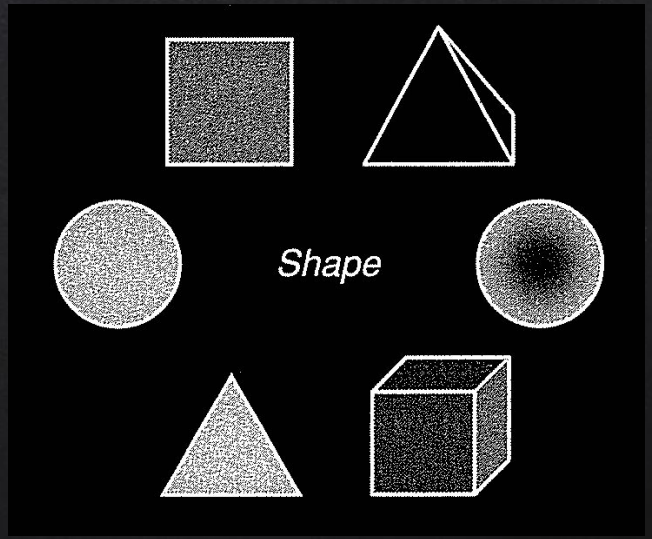
É possível substituir operações concretas?

SIM!

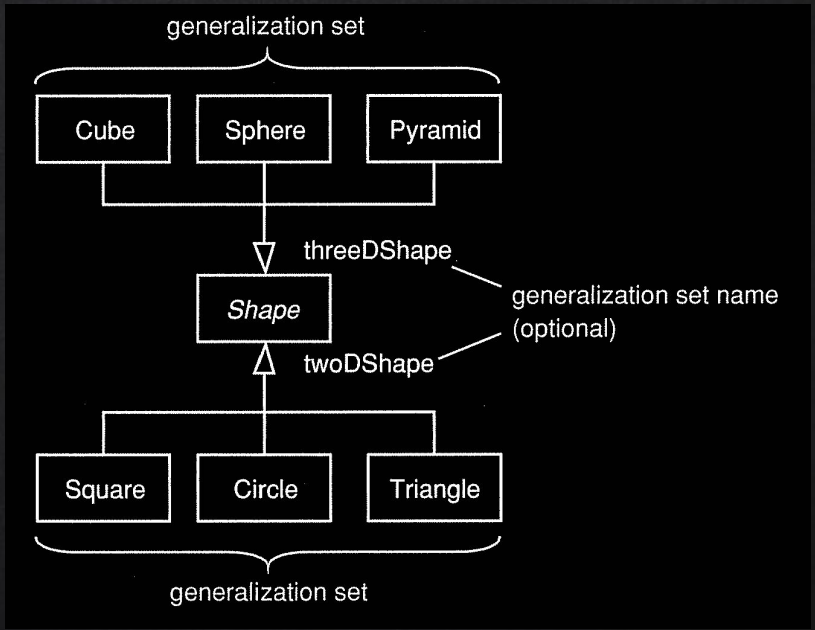
Mas, por favor, não o faça.
É um péssimo estilo!

A generalização define subclasses de partição de acordo com uma regra

Como se dividem as seguintes formas?



Podemos usar conjuntos de generalização para separar os dois subconjuntos (formas 2D e 3D)

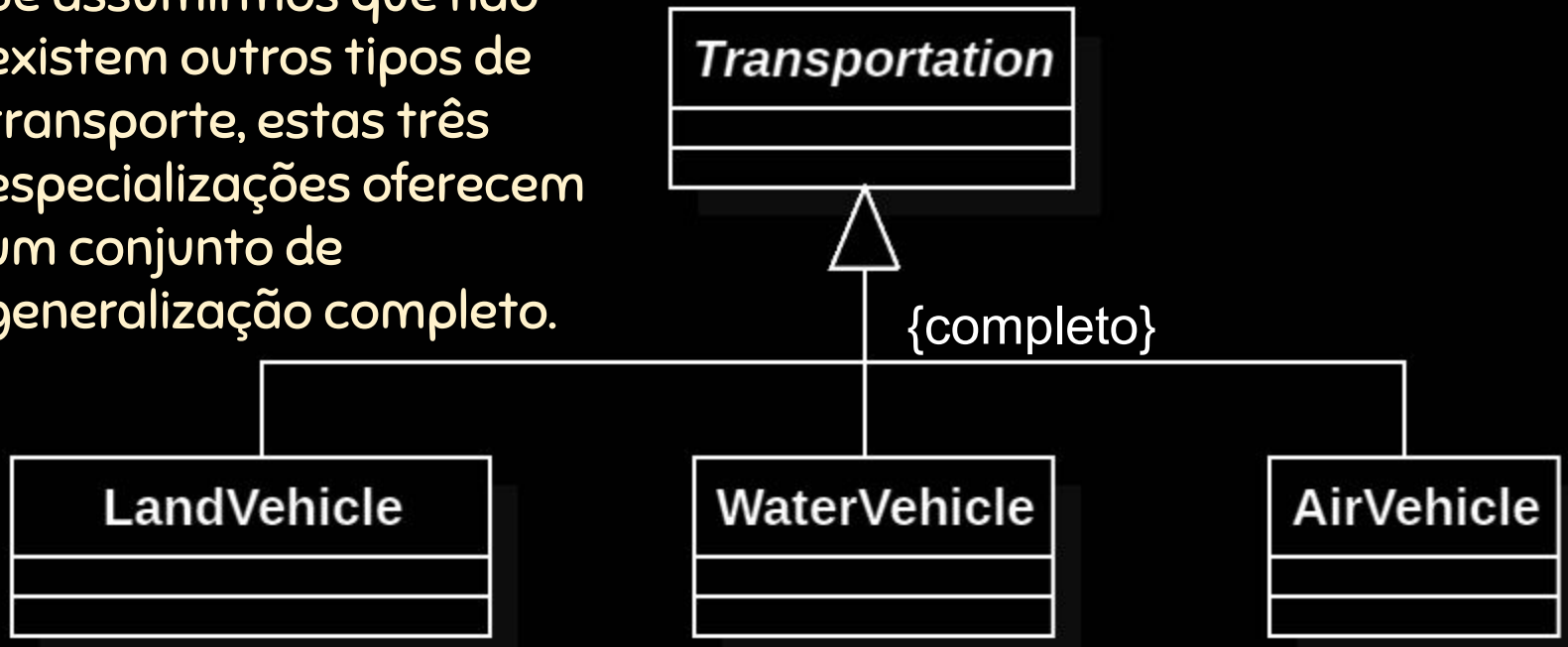


Os conjuntos de generalização podem ser restringidos

- {complete}
  - subclasses cobrem todas as possibilidades
- {incomplete}
  - podem existir outras subclasses ainda não modeladas
- {disjoint}
  - um objecto pode ser uma instância de um e apenas um dos membros do conjunto de generalização (mais comum)
- {overlapping}
  - um objecto pode ser uma instância de mais de um dos membros do conjunto de generalização (requer herança múltipla ou classificação múltipla)

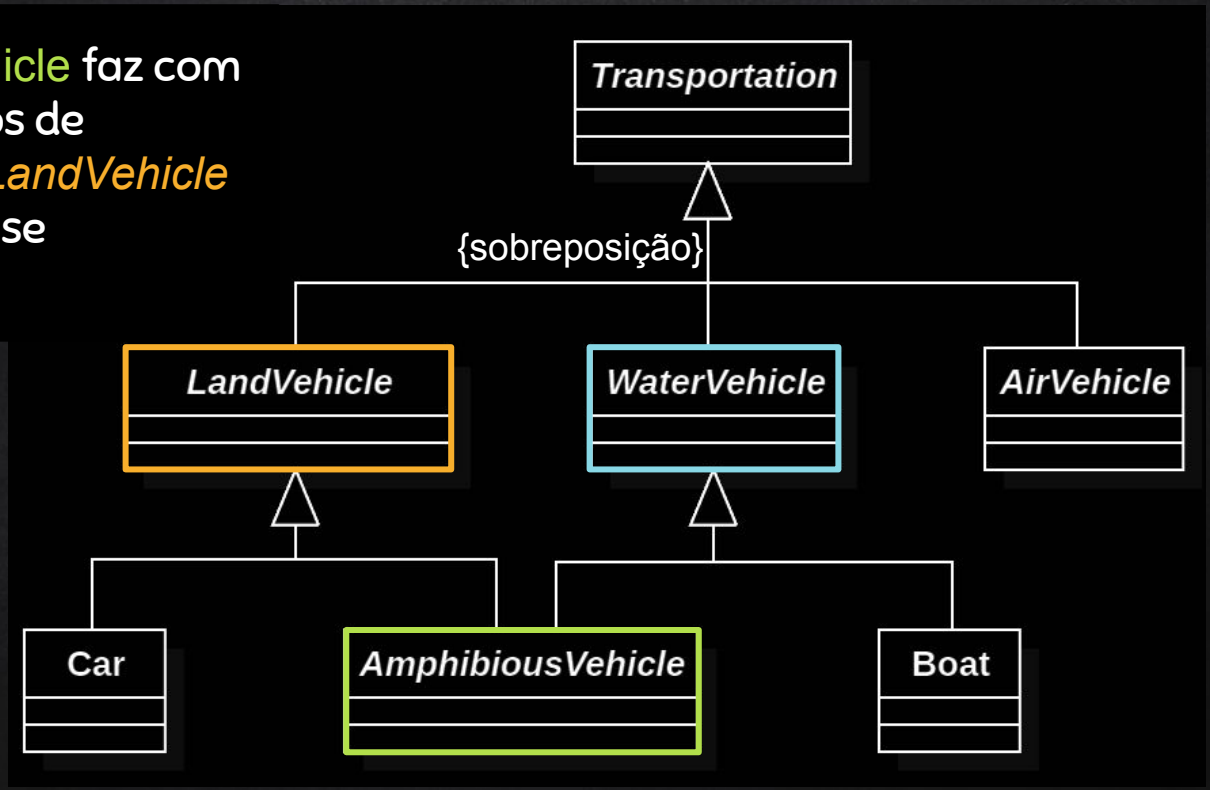
{complete} Conjunto de generalização

Se assumirmos que não existem outros tipos de transporte, estas três especializações oferecem um conjunto de generalização completo.



{overlapping} Conjuntos de generalização

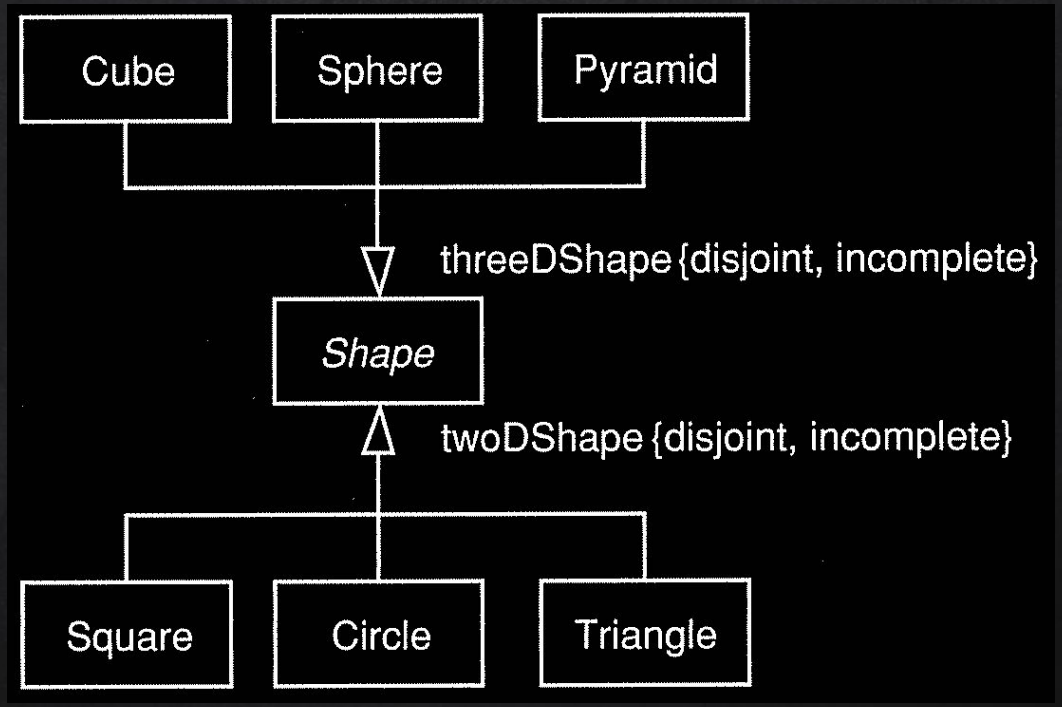
AmphibiousVehicle faz com que os conjuntos de generalização *LandVehicle* e *WaterVehicle* se sobreponham.



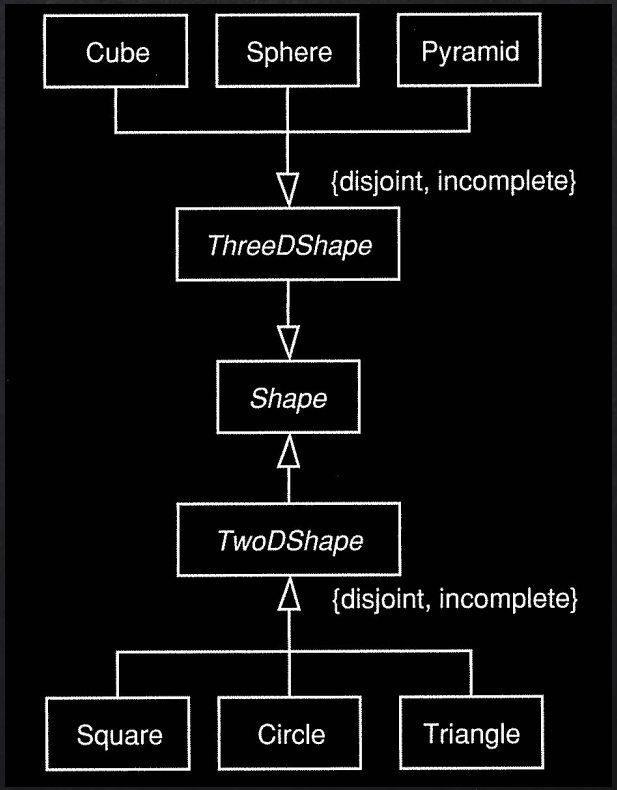
Os Conjuntos de generalização podem ser combinados

Restrição	Conjunto completo	Os membros do conjunto podem ter instâncias em comum
{incomplete, disjoint} – padrão	N	N
{complete, disjoint}	S	N
{incomplete, overlapping}	N	S
{complete, overlapping}	S	S

Aplicação de conjuntos de generalização a Shapes

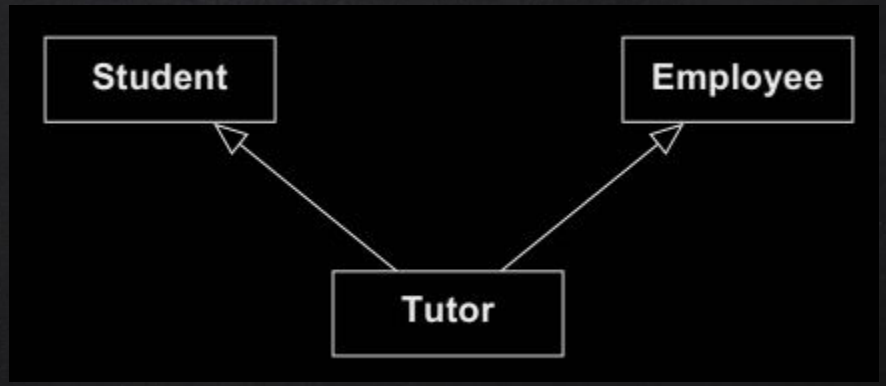


Decomposição de conjuntos de generalização em novas classes na hierarquia de herança



Herança múltipla

Uma classe pode ter herança múltipla, ou seja, várias superclasses

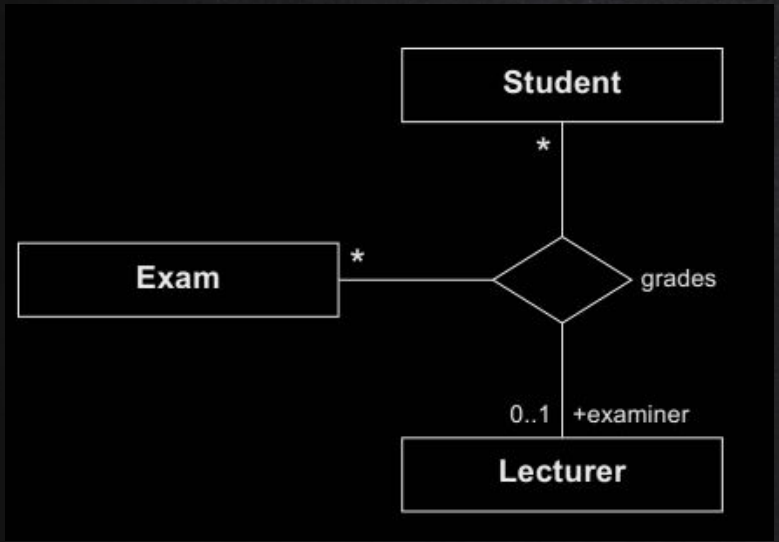


Benefícios e desafios da herança

- Benefícios da herança
  - Mecanismo de abstração/classificação de entidades
  - Mecanismo para reutilização
- Desafios
  - Só podemos compreender as classes se conhecermos as suas superclasses
  - Às vezes, o grafo de herança não é compatível com eficiência

Associações N-árias

Uma associação N-ária é representada por um losango ligado a cada um dos 3 (ou mais) classificadores

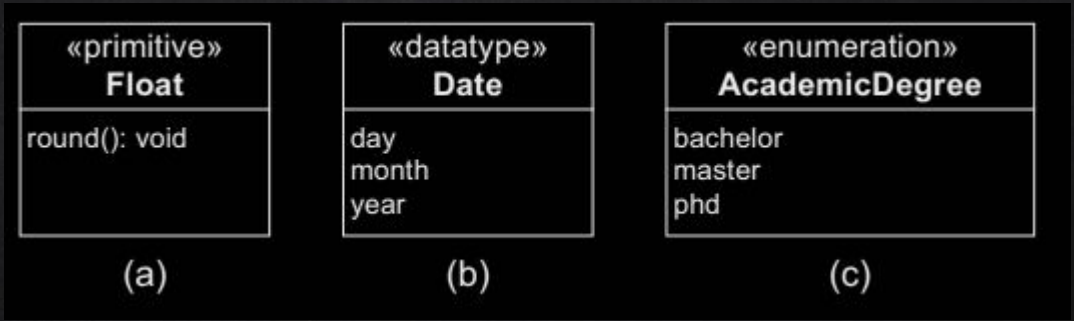


Os multiplicadores definem quantos objectos de uma função/classe podem ser atribuídos a um tuplo fixo (n – 1) de objectos das outras funções/classes

- Um aluno faz um exame sem professor (ou seja, não faz o exame) ou com exatamente um professor.
- Um exame com um professor pode ser feito por qualquer número de alunos
- Um aluno pode ser avaliado por um professor em qualquer número de exames.

Tipos de dados

Em UML, um tipo de dados é visualizado da mesma forma que uma classe, com a diferença de que o nome do tipo de dados é anotado com a palavra-chave adicional <<primitive>>, <<datatype>> ou <<enumeration>>

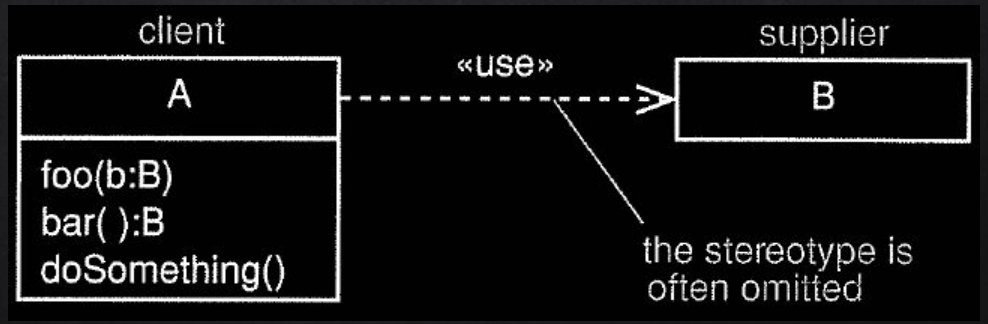


Tipos de associação avançados

Dependência «use»

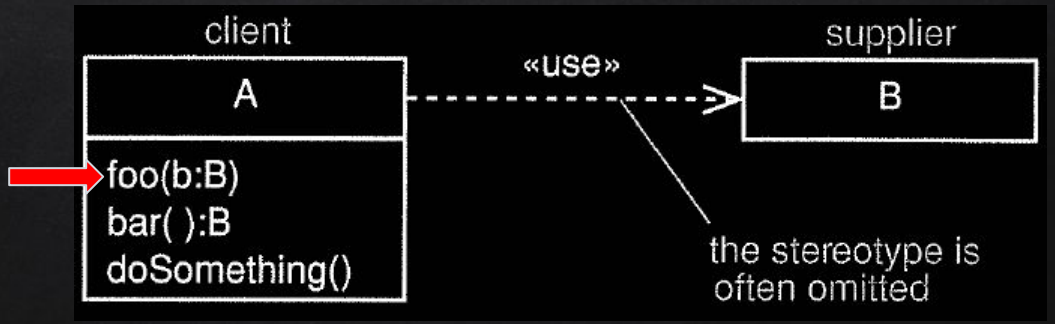
O cliente utiliza os serviços do fornecedor para implementar alguns dos comportamentos do cliente

- Uma operação da classe A precisa de um parâmetro da classe B
- Uma operação da classe A devolve um objecto da classe B
- Uma operação da classe A faz algo que utiliza um objecto da classe B
  - Por exemplo, doSomething() pode criar uma variável local do tipo da classe B



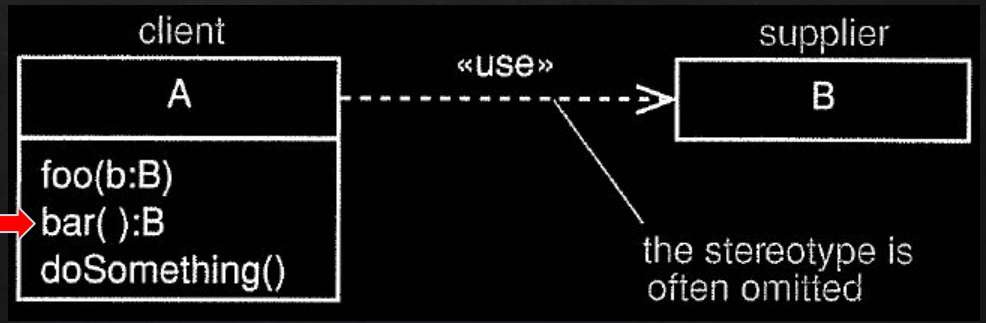
Dependência «use»: Uma operação da classe A precisa de um parâmetro da classe B

```
class A {
    ...
    void foo(B b) {
        ...
    }
}
```



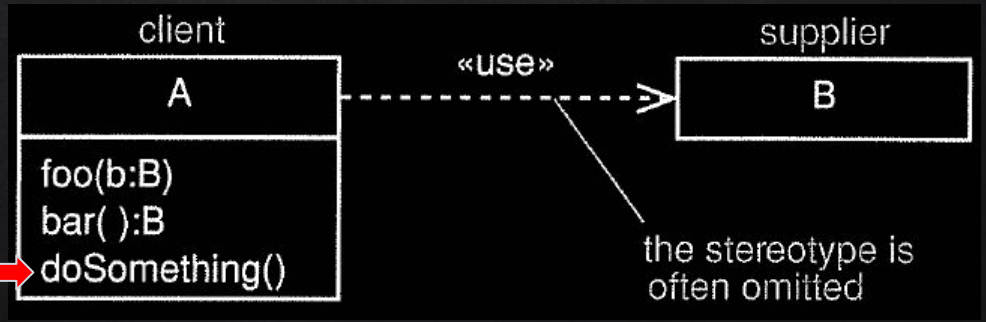
Dependência «use»: Uma operação da classe A devolve um objecto da classe B

```
class A {
    ...
    B bar() {
        ...
    }
}
```



Dependência «use» : Uma operação da classe A faz algo que usa um objecto da classe B

```
class A {
    ...
    void doSomething() {
        B myB = new B();
        // use myB in some way
        ...
    }
}
```



Agregação e composição

Agregação e composição

Agregação

Um tipo de relação flexível entre objectos (por exemplo, um computador e os seus periféricos)



Composição

Um tipo de relação muito forte entre objectos (por exemplo, uma árvore e as suas folhas)

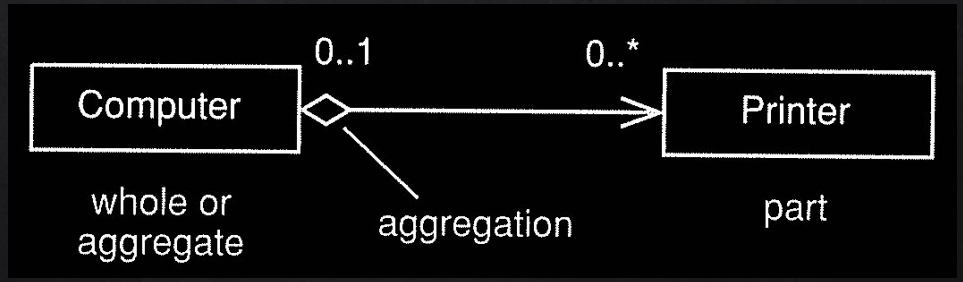


Agregação

A agregação é uma relação entre o todo e as partes

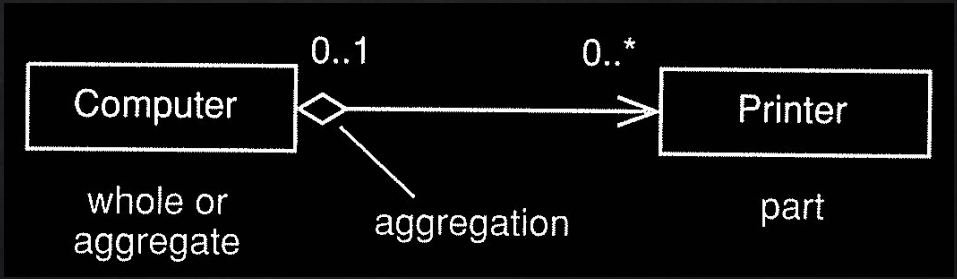
Um tipo de relação entre o todo e as partes em que

- O agregado é composto por muitas partes
- O agregado utiliza os serviços das suas partes
- O agregado desempenha o papel do todo
- Se apenas existir navegabilidade do todo para as partes, as partes não têm consciência de que fazem parte do todo



Semântica do exemplo de agregação

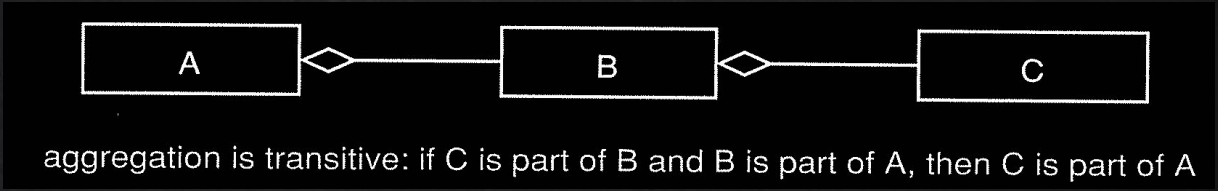
- Um computador pode estar ligado a 0 ou mais impressoras
- Em qualquer momento, uma impressora está ligada a, no máximo, um computador
- Ao longo do tempo, muitos computadores podem usar uma impressora
- A impressora pode existir mesmo que não existam computadores conectados
- A impressora é, na verdade, independente do computador



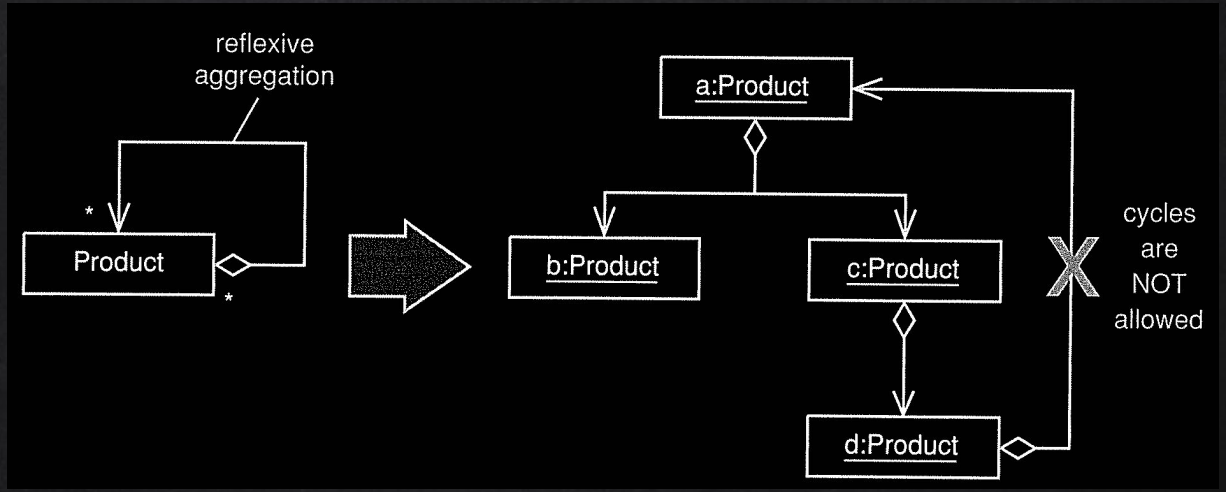
Resumo da semântica da agregação

- O agregado pode, por vezes, existir independentemente das partes, outras vezes não
- As partes podem sempre existir independentemente do agregado
- O agregado está, de alguma forma, incompleto se algumas das partes estiverem em falta
- É possível que várias agregações tenham propriedade partilhada das partes

A agregação é transitiva

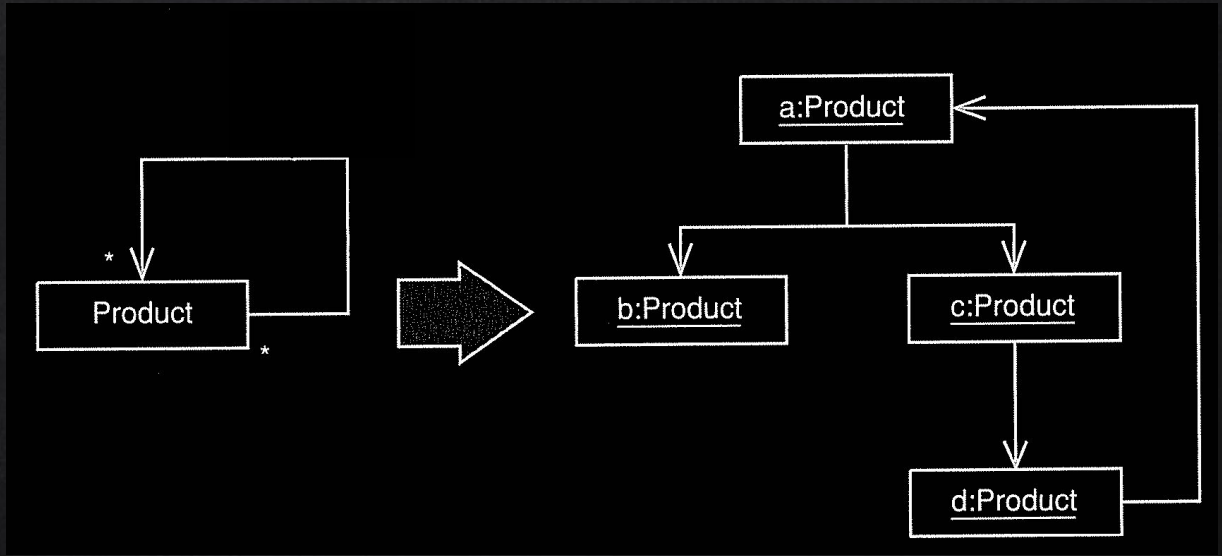


A agregação é assimétrica: nenhum objecto pode fazer parte de si mesmo



Este modelo pressupõe que, embora os produtos possam ser compostos por outros produtos, todos eles são produtos diferentes e a restrição de assimetria é preservada.

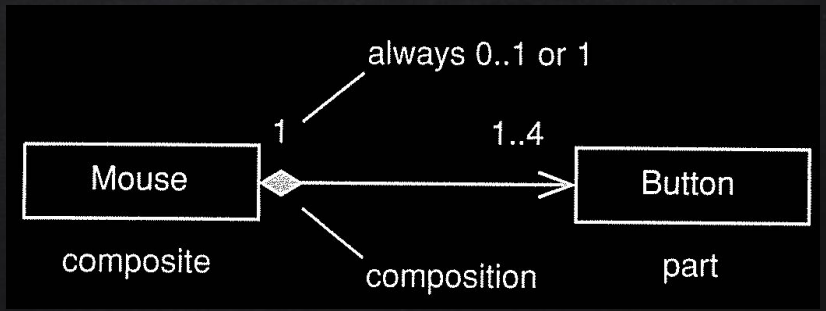
Isto é diferente da Associação, que é simétrica – um objecto pode estar associado a si mesmo



Composição

A composição é uma forma mais forte de agregação

- Tal como a agregação, é uma relação entre o todo e as partes, transitiva e assimétrica
- Ao contrário da agregação, na composição as partes não têm vida independente fora do todo
- Na composição, cada parte pertence a, no máximo, um e apenas um todo
- Se o objecto for destruído, todas as suas partes são destruídas



Semântica da composição

- As partes só podem pertencer a um objecto composto de cada vez – não existe a possibilidade de partilhar a propriedade de uma parte
- O objecto composto tem responsabilidade exclusiva pela disposição de todas as suas partes – isto significa que tem responsabilidade pela sua criação e destruição
- O objecto composto pode libertar partes, desde que a responsabilidade por elas seja assumida por outro objecto
- Se o objecto composto for destruído, também deve destruir todas as suas partes ou transferir a responsabilidade para outro objecto

Agregação vs. composição

Agregação

É possível formar hierarquias e redes de agregação reflexivas – um objecto pode fazer parte de mais de um agregado

Composição

Só podem existir hierarquias de composição reflexivas – um objecto só pode fazer parte de um objecto composto em qualquer momento



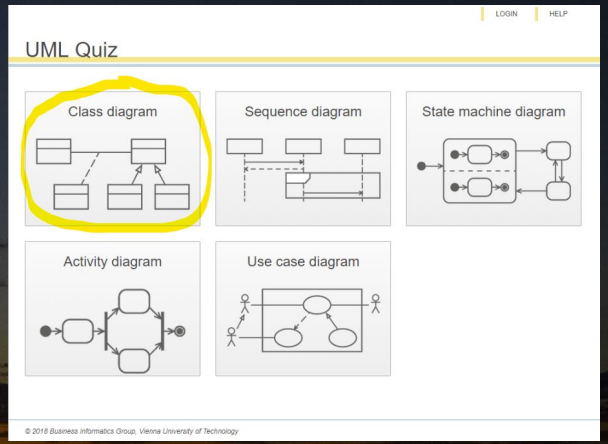


Composição e atributos: escolhem-se os mais adequados para aumentar a clareza, utilidade e legibilidade do modelo

- Uma parte de um objecto composto é equivalente a um atributo
  - Um atributo pode ser visto como uma relação de composição entre a classe composta e a classe do atributo
- No entanto, usamos as duas formas de expressar a mesma coisa, muitas vezes nos mesmos diagramas. Porquê?
  - Os atributos podem ser de tipos de dados primitivos. Poder-se-íam estereotipar com <<primitive>>, mas isto sobrecarrega o modelo. Por isto, os atributos de tipo primitivo são sempre modelados como atributos
  - Existem várias classes utilitárias, como Time, Date ou String, que são usadas com muita frequência. Novamente, embora estes tipos também possam ser modelados, esta modelação também sobrecarrega o diagrama. Assim, também são modelados como atributos.

Perceberam os Diagramas de Classe? Testem os conhecimentos em: https://elearning.uml.ac.at/





Por que é que os programadores não gostam da natureza?

Tem demasiados bugs.

Referências

UML 2 and the Unified Process: Practical Object-Oriented Analysis and Design

2ª Edição, Jim Arlow and Ila Neustadt, Pearson Education

Capítulos 7, 8, 9 e 10

